



CS244 Advanced Programming Applications

Dr Walid M. Aly

Lecture 3

Object-Oriented Relationships

Group home page:

<http://groups.yahoo.com/group/JAVA-CS244/>



Object-Oriented Relationships

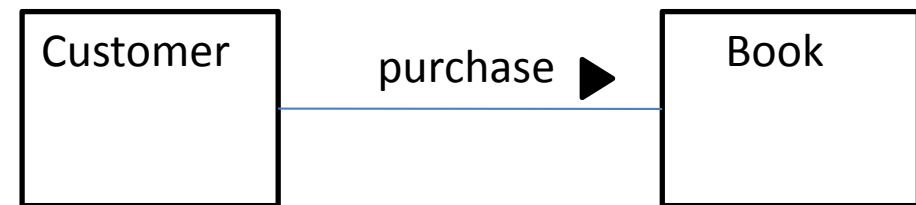
- Object oriented programs usually consisted of a **number** of classes
- Only **one** of theses classes will have the main method
- Classes have different relationships (associations)
- **association** defines a **relationship** between classes of objects which allows one object instance to cause another to perform an action on its behalf



Implementing Object-Oriented Relationships : **Association**

One way of implementation is by having object as a **method argument**

A customer can purchase a book



```
public class Book
{
    private double price;
    public double getPrice()
    {
        return price;
    }
}
```

```
public class Customer
{
    ....
    public void purchase(Book book)
    {
        book.getPrice();
        /////
    }
    .....
}
```



Example Association by passing objects as arguments

```
public class Book{
    private double price;
    private String title;
    public Book(double price, String title)
    {
        this.price=price;
        this.title=title;
    }
    public double getPrice()
    {
        return price;
    }
    public String getTitle()
    {
        return title;
    }
}
```

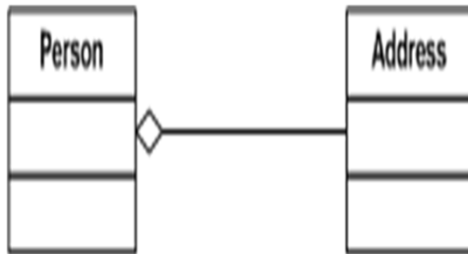
```
public class Customer{
    private double purchaseAmount;
    public void purchase(Book book)
    {
        purchaseAmount +=book.getPrice();
    }
    public double getPurchaseAmount()
    {
        return purchaseAmount ;
    }
}
```

```
public class Main{
    public static void main(String [] args){
        Customer cutomer1=new Customer ();
        Book book1=new Book(100,"Java");
        Book book2=new Book(200,"C#");
        cutomer1. purchase(book1);
        cutomer1. purchase(book2);
        System.out.println(cutomer1.getPurchaseAmount());
    }
}
```



Aggregation

- Aggregation is a type of *association*.
- A connection between two objects where one has a field that refers to the other.
- Aggregation is referred to as **weak has a**.
- **has a** relationship is implemented by the having object as **member variables**
- The lifetime of objects are not associated, objects created separately.



“A person **has a** lawyer “

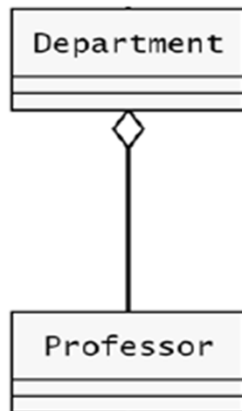
```
class Person
{

```

```
class Lawyer
{
    public void sue()
}

```

“A department **has a** professor“



```
public class Person{
    private Lawyer lawyer;
    Person (Lawyer lawyer){
        this.lawyer=lawyer;
    }
    .....
    lawyer.sue()
}

```

```
class Test
{
    ....
    Lawyer mylawyer=new Lawyer ();
    Person p1=new Person(myLawyer);
    Person p2=new Person(myLawyer);
}

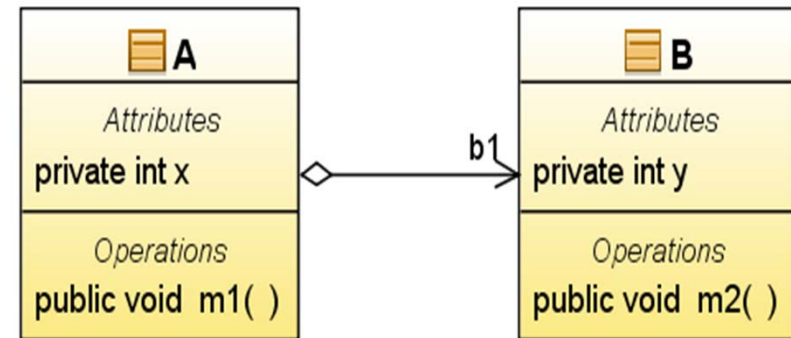
```



UML Representation for Aggregation

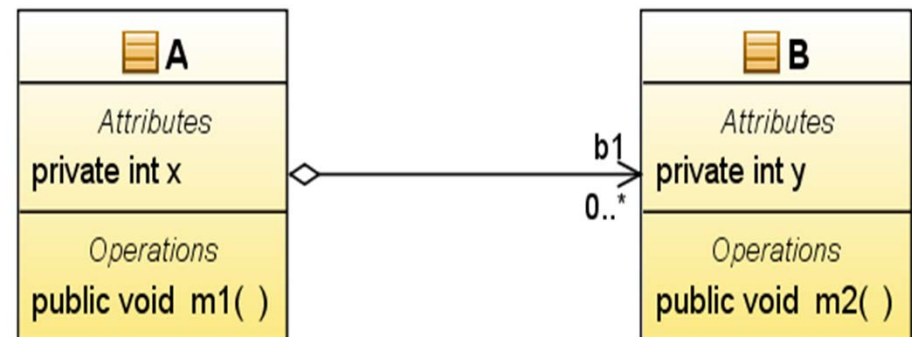
```
public class A {  
    private B b1;  
    private int x;  
    public void m1(){}  
}
```

```
public class B {  
    private int y;  
    public void m2(){}  
}
```



```
public class A {  
    private B [] b1;  
    private int x;  
    public void m1(){}  
}
```

```
public class B {  
    private int y;  
    public void m2(){}  
}
```





Case Study:

Students borrowing books from more than one library



```
class Student {  
    private String name=null;  
    private String email=null;  
  
    public Student() {  
        name = "Unassigned";  
        email = "Unassigned";  
    }  
    public String getEmail( ) {  
        return email;  
    }  
    public String getName( ) {  
        return name;  
    }  
    public void setName(String studentName) {  
        name = studentName;  
    }  
    public void setEmail(String address) {  
        email = address;  
    }  
}
```

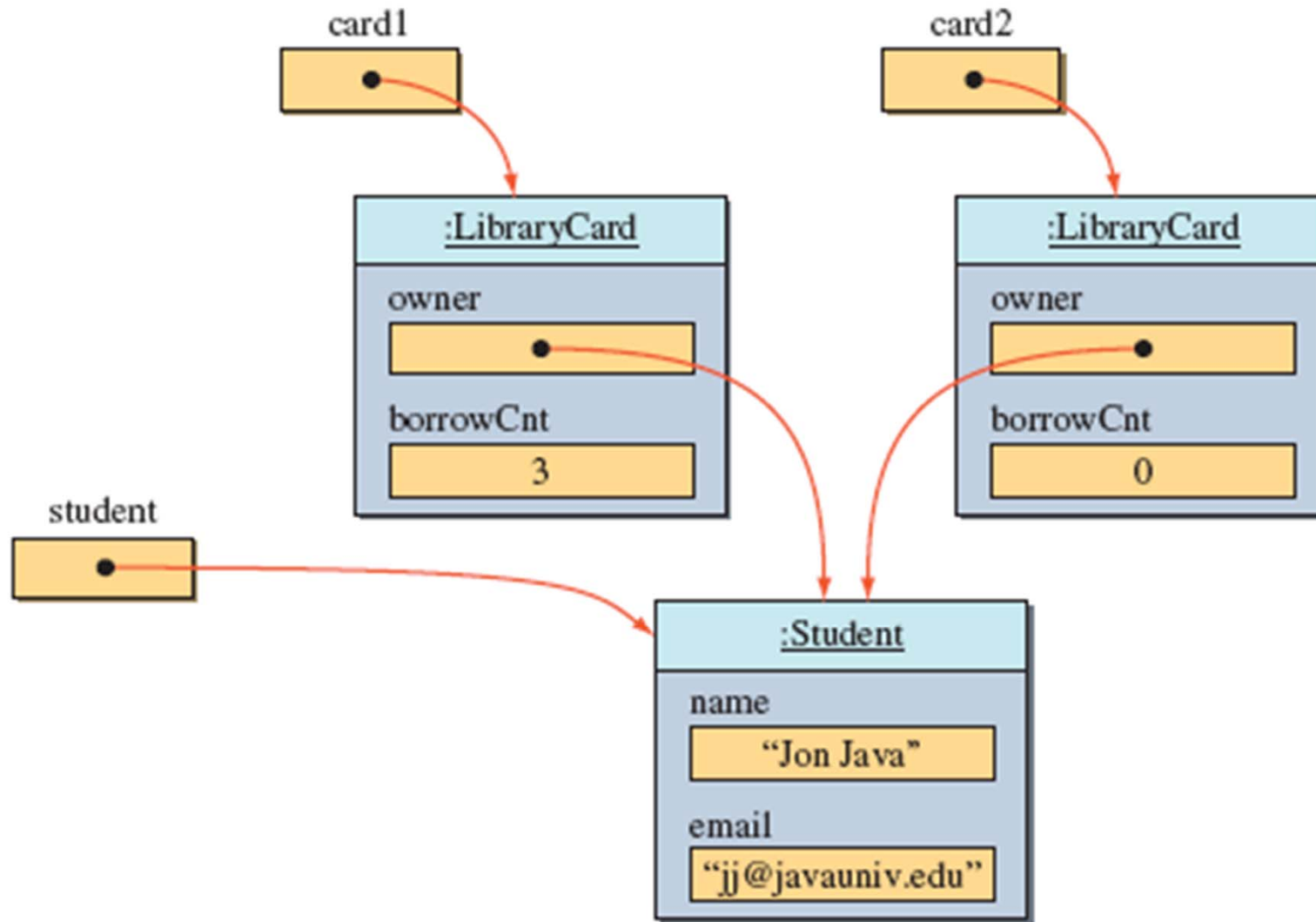



```
class LibraryCard {  
    private Student owner;  
    private int borrowCnt;  
  
    public void checkOut(int numOfBooks) {  
        borrowCnt = borrowCnt + numOfBooks;  
    }  
  
    public String getOwnerName( ) {  
        return owner.getName( );  
    }  
    public int getNumberOfBooks( ) {  
        return borrowCnt;  
    }  
    public void setOwner(Student student) {  
        owner = student;  
    }  
    //Returns the string representation of this card  
    public String toString( ) {  
        return "Owner Name: " + owner.getName( ) + "\n" + "    Email: " +  
owner.getEmail( ) + "\n" + "Books Borrowed: " + borrowCnt;  
    }  
}
```



```
class Librarian {  
  
    public static void main( String[] args ) {  
        Student student;  
        LibraryCard card1, card2;  
  
        student = new Student( );  
        student.setName("Jon Java");  
        student.setEmail("jj@javauniv.edu");  
  
        card1 = new LibraryCard( );  
        card1.setOwner(student);  
        card1.checkOut(3);  
        card2 = new LibraryCard( );  
        card2.setOwner(student); //the same student is the owner of the second card, too  
        System.out.println("Card1 Info:");  
        System.out.println(card1.toString() + "\n");  
        System.out.println("Card2 Info:");  
        System.out.println(card2.toString() + "\n");  
    }  
}
```

```
Card1 Info:  
Owner Name:      Jon Java  
Email:           jj@javauniv.edu  
Books Borrowed:  3  
  
Card2 Info:  
Owner Name:      Jon Java  
Email:           jj@javauniv.edu  
Books Borrowed:  0  
  
Press any key to continue . . .
```





Case Study: **Students registered in a course**

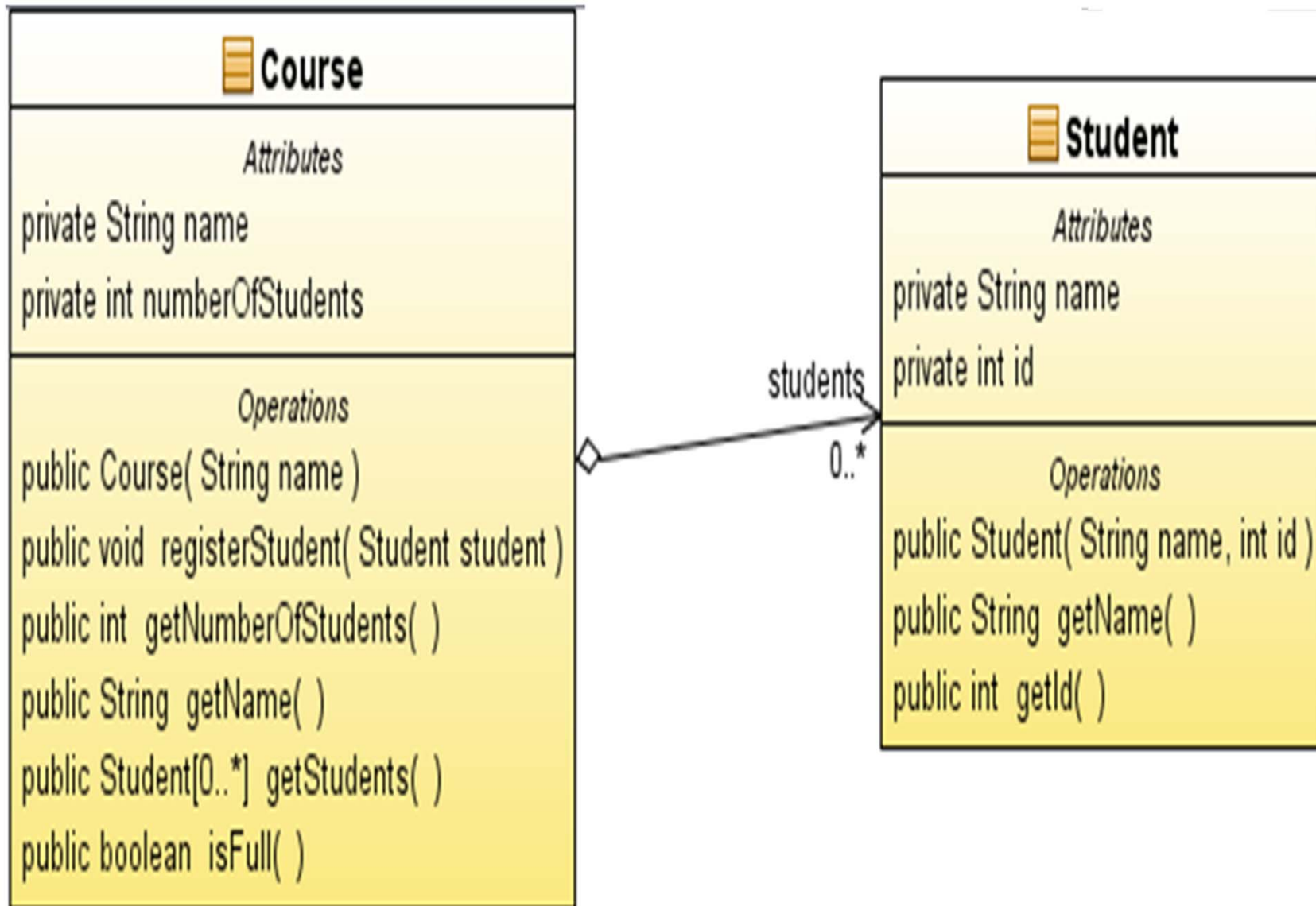


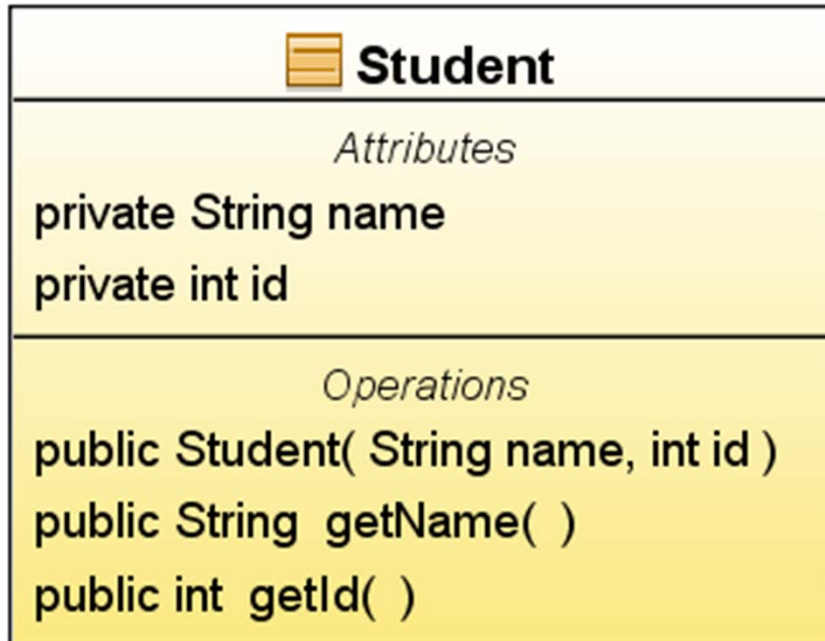
Case Study : Students registered in a course

➤Design a simple registration system that allows Student to register in a course using 2 classes :**class Student** & **class Course**

- Each student has a name and an id variables.
- Each object of class Student is initialized using values of name and id passed to constructor.
- Class Student has accessor methods for its instance variables

- Each **Course** has a **name**, and a variable **numberOfStudent** representing the number of registered students, a Course can have a maximum number of 10 students registered in it .
 - Class course store the registered students in **students** which is an array of type Student, when a student register in a course , he is added to the array.
 - Each object of class **Course** is initialized using the title.
- Class **Course** has the following methods :
- Method **boolean isFull()** , return true if the course is full .
 - accessor method for the **title** and **numberOfStudent** field.
 - method **register Student(Student student)**: if the course is not full, register a student in course





```
public class Student {  
    private String name;  
    private int id;  
  
    public Student(String name,int id)  
    {  
        this.name=name;  
        this.id=id;  
    }  
    public String getName()  
    {  
        return name;  
    }  
    public int getId()  
    {  
        return id;  
    }  
}
```

```

public class Course {
    private String name;
    private Student[] students = new Student[10];
    private int numberOfStudents;

    public Course(String name) {
        this.name = name;
    }
    public void registerStudent(Student student) {
        if (isFull())
        {
            return;
        }
        students[numberOfStudents] = student;
        numberOfStudents++;
    }
    public int getNumberOfStudents() {
        return numberOfStudents;
    }
    public String getName() {
        return name;
    }
}

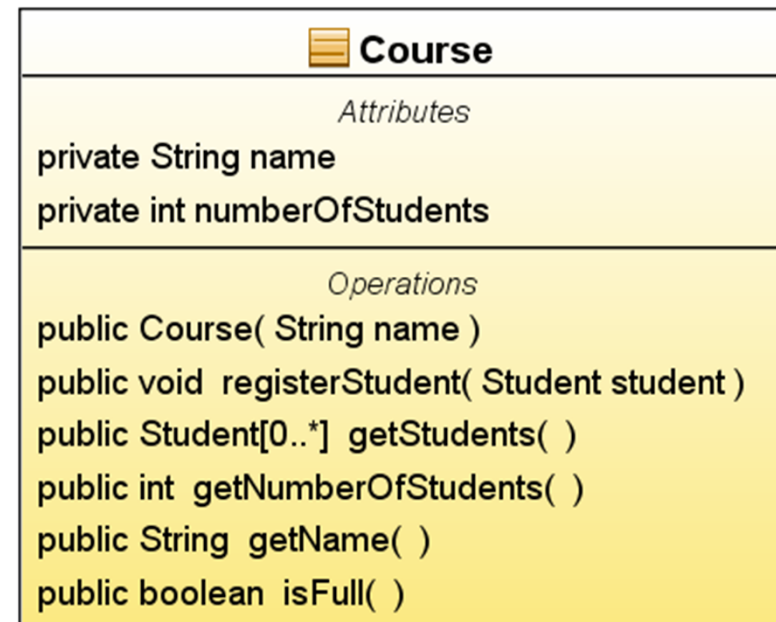
```

```

    public Student[] getStudents() {
        return students;
    }

    public boolean isFull()
    {
        if (numberOfStudents==10)
            return true;
        else
            return false;
    }
}

```




```
public class Test{  
    public static void main(String[] args) {
```

```
        Course course1 = new Course("Data Structures");  
        Student student1=new Student("Peter Jones",4);
```

```
        course1.registerStudent(student1);  
        course1.registerStudent(new Student("Brian Smith",7));  
        course1.registerStudent(new Student("Anne Kennedy",9));
```

```
        Course course2 = new Course("Database Systems");
```

```
        course2.registerStudent(student1);
```

```
        course2.registerStudent(new Student ("Steve Smith",7));
```

```
            int num1= course1.getNumberOfStudents();
```

```
            System.out.println("Number of students in course1: " +num1);
```

```
            Student[] students = course1.getStudents();
```

```
            for (int i = 0; i < course1.getNumberOfStudents(); i++){
```

```
                String name=students[i].getName() ;
```

```
                System.out.println(name);
```

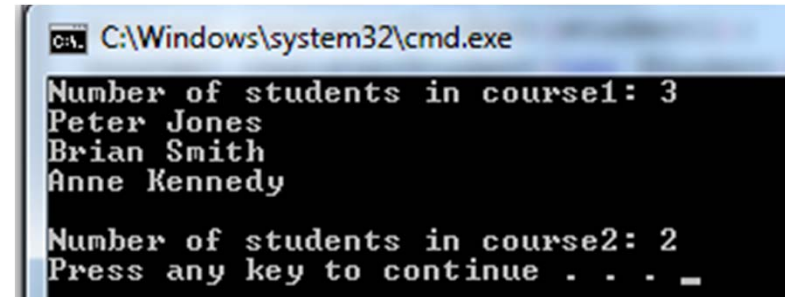
```
            }
```

```
            System.out.println();
```

```
            System.out.println("Number of students in course2: " + course2.getNumberOfStudents());
```

```
        }
```

```
    }
```



```
C:\Windows\system32\cmd.exe  
Number of students in course1: 3  
Peter Jones  
Brian Smith  
Anne Kennedy  
Number of students in course2: 2  
Press any key to continue . . . _
```

Composition

- A **composition** is a strong type of aggregation.
- **Each component in a composite can belong to just one whole.**
- Composition defines the relation “**owns a**” / “**part-of relationship**”/ **strong has a**
- The lifetime of objects are associated, object of the part is created within owner.

The CoffeeTable **owns** 4Leg

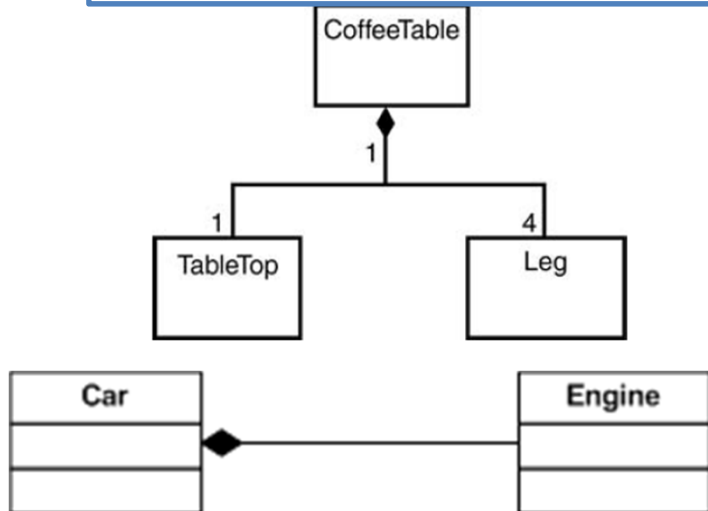


Figure 1 - Composition

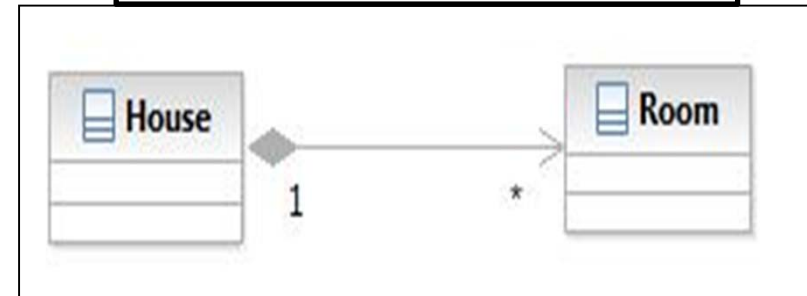
```

public class Engine
{....
}
    
```

```

public class Car{
    Engine e;
    public Car(){
        e=new Engine();
    }
}
    
```

A house **owns** a room



```

public class Test
{
    public static void main (String [] arg)
    {
        Car car1 =new Car();
    }
}
    
```



Inheritance: **is a** relationship

- When creating a class, rather than declaring completely new members, the programmer can designate that the new class **inherit** the **members** of an existing class.
- The existing class is called the **superclass**, and the new class is the **subclass**.
- Inheritance allows the subclass to use the none private members of the super class directly.
- A subclass normally adds its own fields and methods.
- The inheritance relation should be implemented . Only when is a relationship is clear and valid

```
class Loan{}
```

```
class CarLoan extends Loan {}
```

```
class A{  
  int i;  
  int j;  
  void m1(){}  
}
```

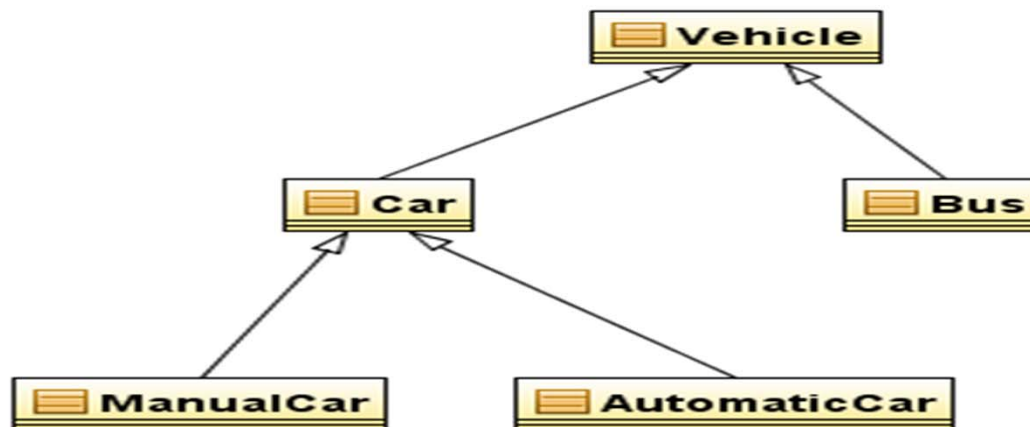
```
class B extends A{  
  int k;  
  void m2(){}  
}
```

```
class Test{  
  public static void main (String [] arg){  
    B b1=new B();  
    b1.i=10;  
    b1.m1();  
  }}
```



Inheritance: **is a** relationship

- Each subclass can become the superclass for future subclasses.
- Multiple inheritance is not allowed.
- In Java, the class hierarchy begins with class **java.lang.Object** which every class in Java directly or indirectly extends (or "inherits from").





A Simple Inheritance case study

```
public class Person{
    private String name;
    public Person ()
    {}
    public void setName (String name){
        this.name=name;
    }
    public String getName(){
        return name;
    }
}
```

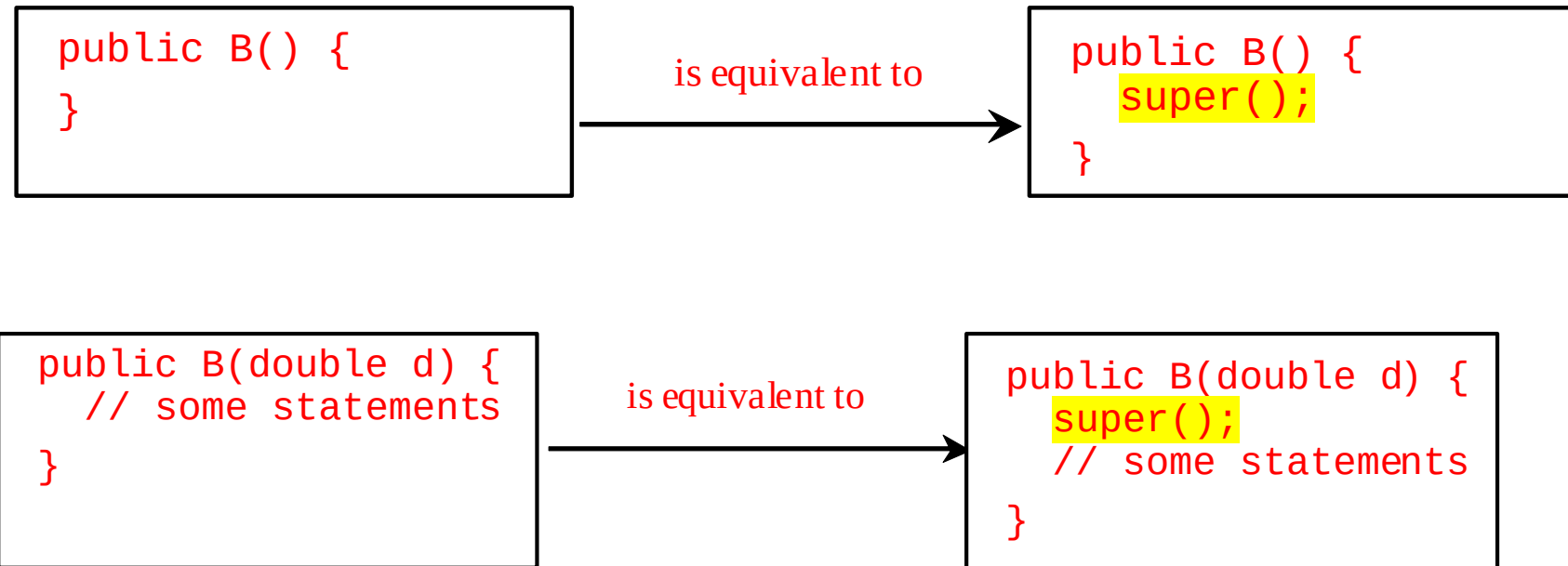
```
public class Student extends Person{
    private int registrationNumber;
    public Student ()
    {}
    public void setRegistrationNumber (int){
        this.registrationNumber=registrationNumber;
    }
    public int getRegistrationNumber(){
        return registrationNumber;
    }
}
```

```
public class Test{
    public static void main (String [] arg)
    {
        Student student1=new Student();
        student1.setName ("Ahmed Hussien");
        student1.setRegistrationNumber(4564);
    }
}
```



Calling a subclass constructor invokes super class constructor

- Creating an instance of sub class means **automatically** creating an instance of the super class by invoking the **default no argument** constructor.
- Java adds automatically **super()** to the first line in the constructor of sub class to invoke the default no argument constructor of the sub class.





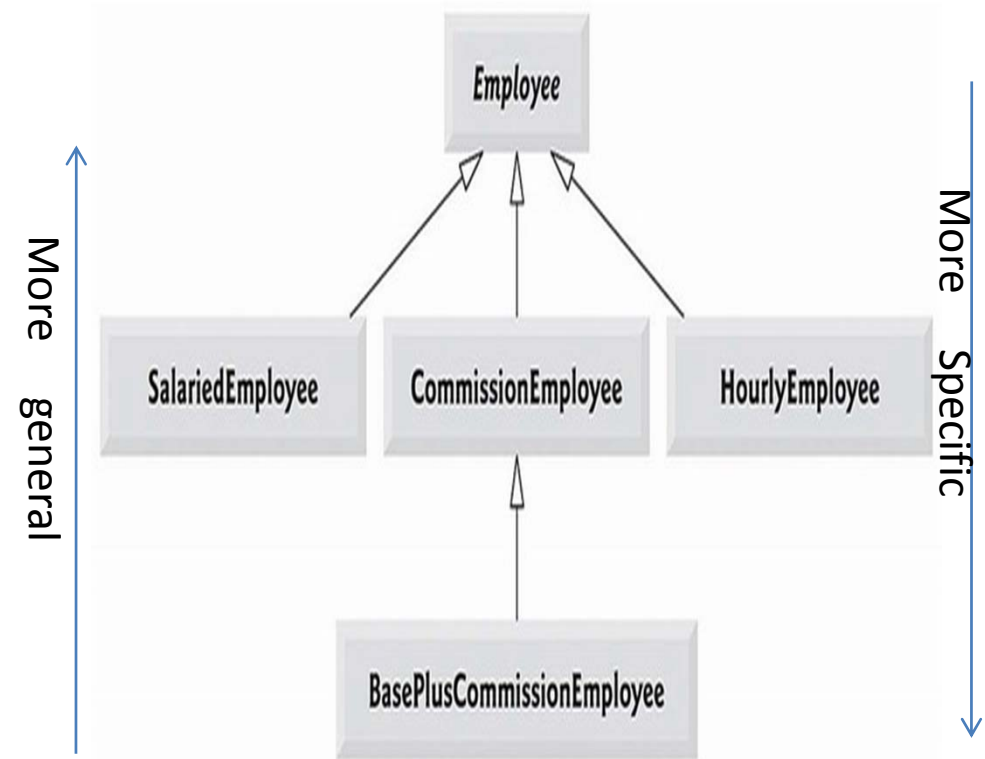
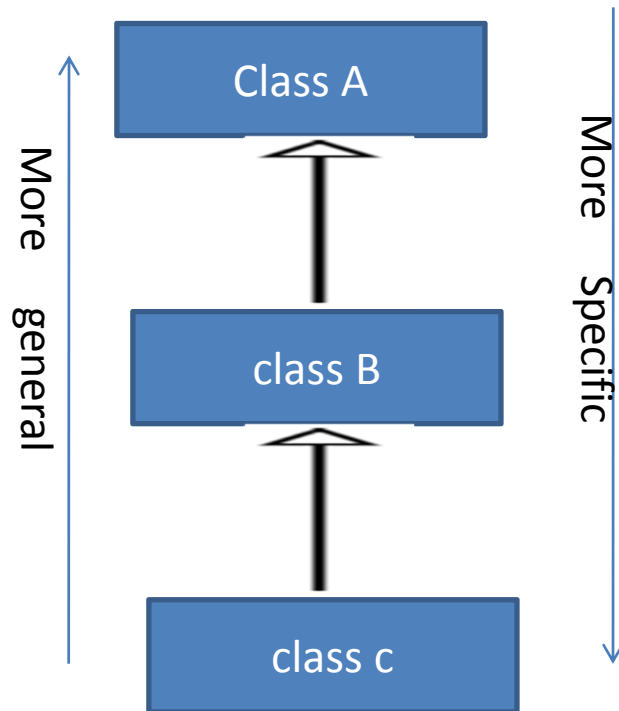
- ❑ You can use the **super** keyword in the subclass constructor to invoke another superclass constructor rather than the default.
- ❑ In a constructor, **super** can only appear at the first line, otherwise will cause **a compile error**.

```
public class Employee{  
    String firstName;  
    String lastName;  
    public Employee( String first, String last)  
    {  
        firstName = first;  
        lastName = last;  
    } ///  
}
```

```
class ComissionEmployee extends Employee  
{  
    private double comission;  
    public ComissionEmployee( String first, String last,double com )  
    {  
        super( first, last);  
        comission=com;  
    }  
}
```



inheritance hierarchy



- In the inheritance hierarchy, classes become more specific and concrete *with each new subclass*. If you move from a subclass back up to a superclass, the classes become more general and less specific.
- Class design should ensure that a superclass contains **common features** of its subclasses.
- More generalization means reaching the **abstraction** level



Modifier : **abstract**

- A method or a class can be declared as **abstract**.
- Access modifiers and abstract keyword can appear in any order.

abstract method

- An abstract method is just a method signature without any implementation.
- abstract methods **cannot** have a body.
- A constructor can never be declared as abstract.
- If a class has **any** abstract methods it **must** be declared as abstract.
- Abstract methods can **not** be static
- Any of the mentioned rules not obeyed results on a **compile error**.
- Non abstract methods are concrete methods

```
public abstract class M
{
    abstract public void m1();
    void m2()
    {
    }
}
```



abstract class

- Sometimes a superclass is so abstract that it cannot have any specific instances. Such a class is referred to as an **abstract class**.
- **no objects can be created** from an abstract class, except from subclasses
- An abstract class **forces** its user to **create a subclass** to execute instance methods
- it is **possible to execute the static** methods of abstract classes, since an object instance is not required in this context
- Subclasses of this abstract class will have to implement the abstract methods of abstract class or declared as abstract themselves.
- Abstract classes are very useful with **polymorphism**
- A class can be declared abstract even if no abstract methods is included.
- A non abstract class is an **concrete class**
- A subclass can override a concrete method in a superclass to declare it abstract.

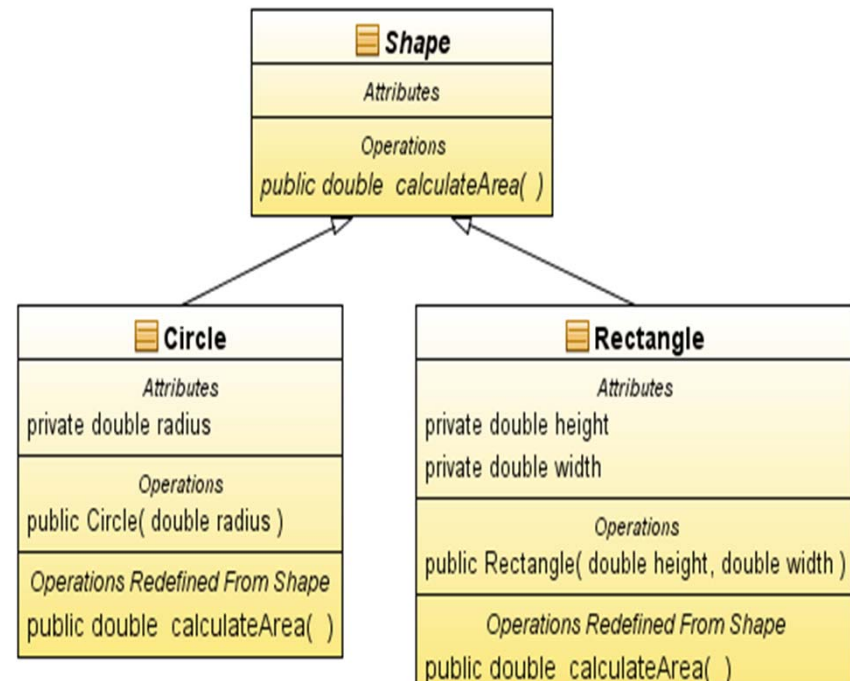


Example : extending abstract class

```
public abstract class Shape {  
    public abstract double calculateArea();  
}
```

```
public class Rectangle extends Shape{  
    private double height, width;  
    public Rectangle( double height,double width)  
    {  
        this.height=height;  
        this.width=width;  
    }  
    public double calculateArea()  
    {  
        return height*width;  
    }  
}
```

```
public class Circle extends Shape {  
    private double radius;  
    public Circle(double radius){  
        this.radius=radius;  
    }  
    public double calculateArea()  
    {  
        return 3.14*radius*radius;  
    }  
}
```



Abstract elements are *italic* in UML Diagrams



Implementing Object-Oriented Relationships : provides a

- An *interface* is a classlike that contains *only* constants and abstract methods and *no* constructors.
- A class *implements* an interface by implementing *all* methods in the interface.
- An interface represents the relation “*provides a*”(promises).
- If a class is declared to implement an interface and does not implement all the methods in the interface , it must be declared *abstract* otherwise *a compile error*
- Variables in interfaces are implicitly *public*, *static* and *final* and should be initialized with a value.
- Methods in interface are implicitly *public* (even if left without a modifier) and *abstract*, **they should only be implemented only by *public* methods.**
- Interfaces can be extended, multiple extension for interfaces is allowed
- A class can implements more than one interface.

Company *provides* Payable

```
public interface Payable
{
    double getPaymentAmount();
}
```

```
class Company implements Payable{
    public double getPaymentAmount()
    {
        ////
    }
}
```



Case study :Payable interface

```
public interface Payable
{
    double getPaymentAmount();
}
```

```
public class Employee implements Payable
{
    private int days;
    private double dayWage;
    Employee(int days,double dayWage )
    {
        this.days=days;
        this.dayWage=dayWage;
    }
    public double getPaymentAmount()
    {
        return dayWage*days;
    }
}
```

```
public class Invoice implements Payable
{
    private int quantity;
    private double pricePerItem;
    Invoice(int quantity,double pricePerItem )
    {
        this.quantity=quantity;
        this.pricePerItem=pricePerItem;
    }
    public double getPaymentAmount()
    {
        return quantity*pricePerItem;
    }
}
```

